

Lecture 12

Async Javascript

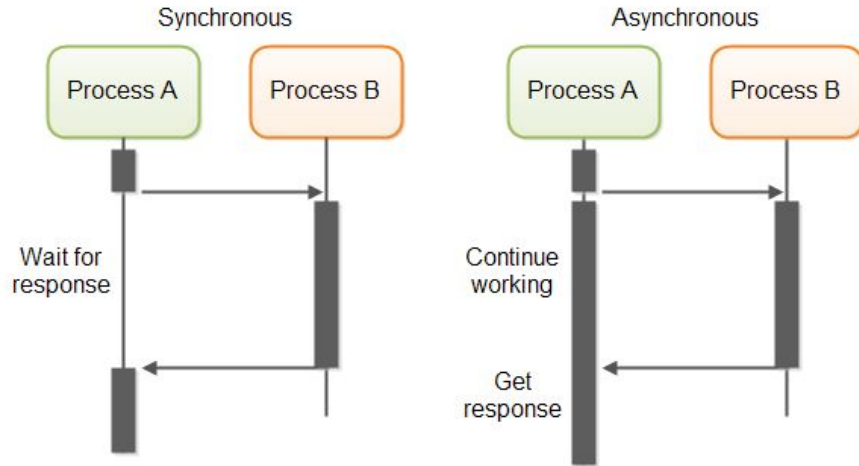
- Callbacks
- Promises
- async/await
- Promise.all()

- AJAX - an example of async
 - fetch() replaces XMLHttpRequest
 - fetch() returns Promise
 - JSON as AJAX payload
(JSON=JavaScript Object Notation)
 - parsing JSON (request.json())
returns a Promise too
- CORS
(Cross-Origin Resource Sharing)

Evolution of async JavaScript

- Asynchronous JavaScript
 - with callbacks before ES6, e.g., in event handlers
 - if heavily nested, callbacks are not too handy
 - more convenient means:
 - Promise ES6 (ES2015)
 - async/await ES8 (ES2017)
 - Promise.all() for parallel processing

Concurrency is hard, two chefs



- sync chef:
slow but all in control



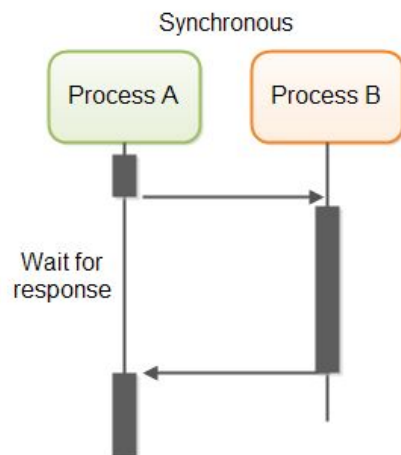
Callback: a function parameter

Synchronous, e.g.,

- self-made f with cb

```
function b() {console.log("moi!");}
function a(cb) {cb()}
a(b);
```

- map(), filter(), reduce()



Asynchronous, e.g.,

- Event handlers

Web APIs:

- built functions for slow ops, e.g.,

- AJAX(XMLHttpRequest)
- File, video/audio
- setTimeout()

```
console.log("Hello");
setTimeout(console.log, 1000, "me");
console.log("it's")
```

- a few Web APIs come in both flavors, sync and async, such as FileReaderSync vs. FileReader
 - async recommended to cause no blocking

Nesting

- pushes indentation to the right
- Example of XMLHttpRequest, older AJAX with callbacks
 - two nestings
 - adequately to give an idea of a 'pyramid of doom'

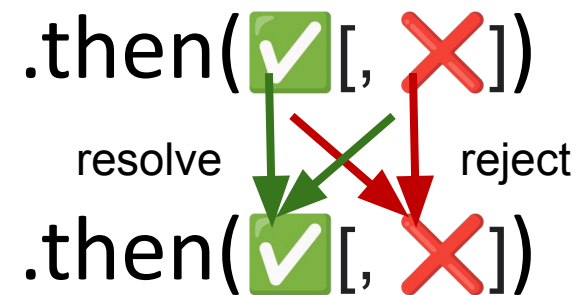
```
const xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
  if (this.readyState == 4 && this.status == 200) {
    const json = this.responseText;
    const ip = JSON.parse(json).ip;
    console.log(ip)
    const xhttp2 = new XMLHttpRequest();
    xhttp2.onreadystatechange = function() {
      if (this.readyState == 4 && this.status == 200) {
        const json2 = this.responseText;
        const ip2 = JSON.parse(json2).ip;
        console.log(`In ${ip===ip2?ip:"controversy"}`);
      }
      xhttp2.open("GET", "https://ifconfig.me", true);
      xhttp2.send();
    }
  }
};
xhttp.open("GET", "https://get.geojs.io/v1/ip/country.json",
true);
xhttp.send();
```

Promises to the rescue

- Promise pends until it is eventually completed, and either
 - fulfilled  or rejected 
 - Promise chain formed of then/catches that return

Promise:

```
new Promise((, )=>  
  .then(,   
  .then(,   
  .then(,   
  .catch()) // == .then(null, 
```



Promise.resolve()/Promise.reject()

```
function resolved(result){ console.log('Resolved'); return "success";}
function rejected(result) {console.error(result); return "fail";}
function rejected2(result) {return Promise.reject("Rejected!!")}
////////////////////////////////////
Promise.resolve("success").then(resolved); //Promise { <state>: "fulfilled", <value>: "success" }
Promise.reject(new Error("fail")).then(null, rejected); //Promise {<fulfilled>: 'fail'}
Promise.reject(new Error("fail")).catch(rejected); //Promise { <state>: "fulfilled", <value>: "fail" }
Promise.reject(new Error("fail")); //Promise { <state>: "rejected", <reason>: Error }
Promise.reject(new Error("fail")).catch(err=>console.log(err)); //Promise { <state>: "fulfilled", <value>: undefined }
Promise.reject(new Error("fail")).catch(rejected2); //Promise { <state>: "rejected", <reason>: "Rejected!!" }
```

async function (1/3)

- Defined by using async keyword in a function

```
async function foo() {} ;           // conventional f  
const bar = async () => {} ; // expression as arrow f
```

- Execution differs from sync functions

- See the event loop:

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/EventLoop>

- Looks like synchronous code

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function

..but async f returns a Promise

async function (2/3)

- informs about anticipated slowness such as AJAX, timeouts or async functions
 - async function must be awaited
 - await pauses the function execution until a promise is resolved
 - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
- behind the syntactic sugar its implementation uses promises and generator functions
 - Generators:
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Iterators_and_Generators

.. to end async chain (3/3)

- async mandates calling function to be async too
- ultimately async chain ends at the main level, inside <script>
- if the chain is desired to be cut earlier: IIFE
 - IIFE, immediately invoked function expression
(async () => { /* ... */ })();

Promise.all()

- Starts parallel Promises and waits for them all to finish
 - total time determined by the slowest Promise
- Returns a single Promise that
 - fulfills when all Promises are resolved
 - resolves as an array
 - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/all

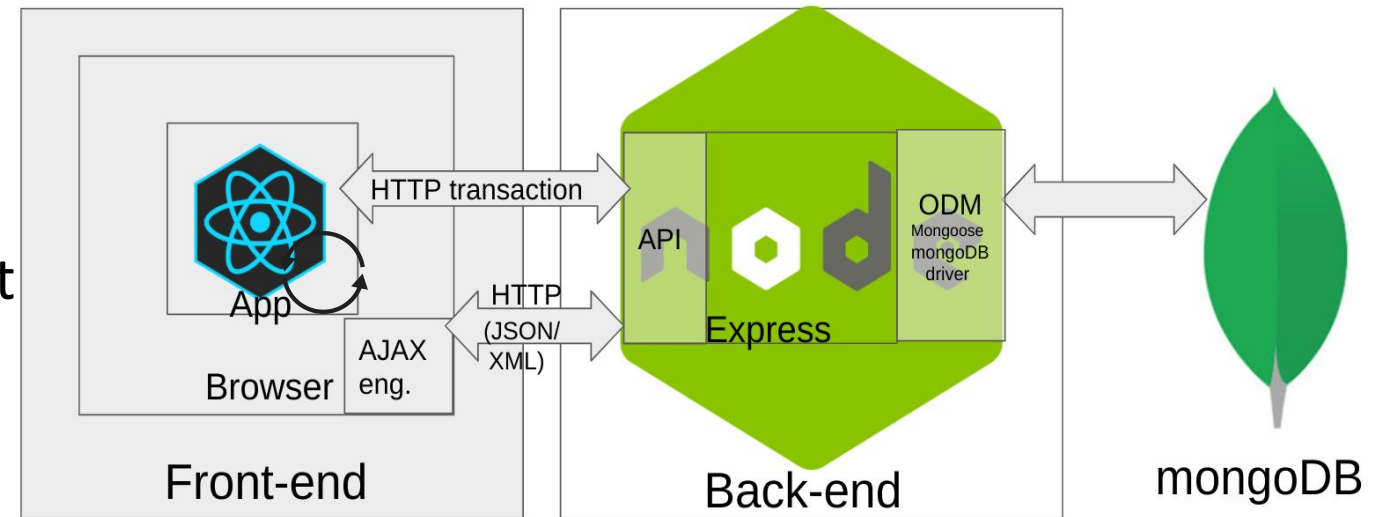
Promise.all() example

```
const pArr = [...Array(5).keys()].map(i=>
  Promise.resolve(`${i}`));
console.log(pArr);
Promise.all(pArr).then(console.log);
```

- range of 5
- map() iterates through them and returns an array of promises
- Promise.all() waits for every promise to resolve, return values are handled in then()

Asynchronous JavaScript and XML (AJAX)

- **HTML** shows the data
- **DOM** updates data presentations dynamically
- **JSON** outnumbered XML as data format
- **XMLHttpRequest (XHR)**
“behind-the-scenes” communication browser ↔ server
- **fetch()** is replacing XHR
being more lightweight and modern,
returns a Promise



fetch()

- A new lightweight AJAX call that replaces old XMLHttpRequest with cleaner API
- The fetch() method takes one mandatory argument, URL, and returns a promise that resolves to the response

```
fetch("https://get.geojs.io/v1/ip/country.json")  
  .then(function(response) {  
    return response.json();  
  })  
  .then(function(data) {  
    console.log(JSON.stringify(data));  
  })  
  .catch(err=>console.log(`fetch err    :${err}`));
```

fetch() returns a Promise

Promise is meant for lengthy asynchronous operations, such as in the previous example:

- a. fetch() executing HTTP transaction (the first return Promise that contains the response body) or
- b. JSON parsing, the second Promise

fetch() with async/await

```
const URL = "https://get.geojs.io/v1/ip/country.json";
const fetchIP = async (URL) => {
  const response = await fetch(URL); //fetch is async
  const json = await response.json(); //json parsing async
  return json.ip;
}
const pFetch = await fetchIP(URL);
console.log(pFetch);
//Promise { <state>: "fulfilled", <value>: "188.67.165.130" }
```


JavaScript Object Notation as payload

- Data is transferred from server to client in a string format:

```
let bookJSstring = '{"topic": "JavaScript", "author": 2011}'; // a JSON string
```

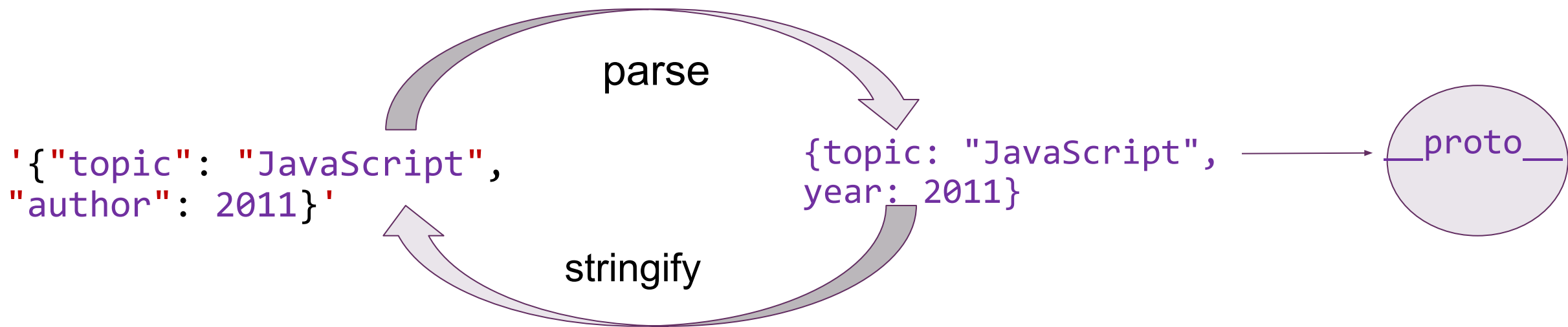
- converted into a JavaScript literal object in the browser

```
let bookObject = {topic: "JavaScript", year: 2011}; // an object
```

- JavaScript has JSON object with built-in methods for doing conversions from and to JSON format

```
let bookObject = JSON.parse(bookJSstring); // convert JSON-string to an object  
let bookJSstring = JSON.stringify(bookObject); // convert object to JSON-string
```

JSON <-> Object

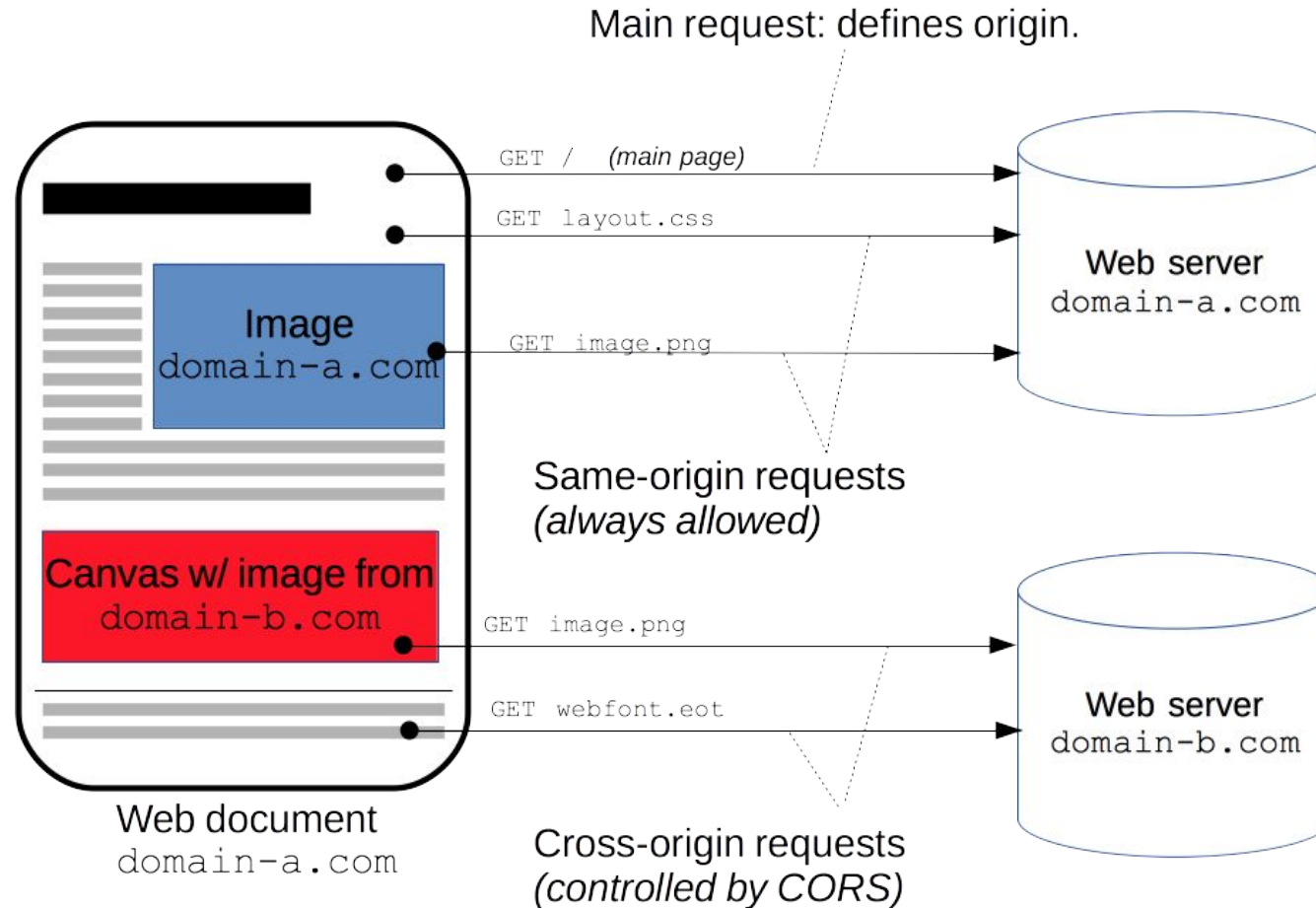


- o **Object literal** is a format that stores data as key-value pairs
 - an object contains a reference to its proto
- o **JSON** is a string, language-agnostic format for storing and transferring data based on the objects
 - in HTTP the content type is `application/json`

CORS (Cross-Origin Resource Sharing)

- A standard to
 - more secure data transfer between a browser and a server
 - enable cross-domain requests as relaxation of same-origin-policy (SOP)
- a browser
 - adds Origin header to the request
 - enables setting the cors header in `fetch()/XMLHttpRequest` object (by default {mode: 'cors'}, for opaque response {mode: 'no-cors'})
 - incorporates new HTTP request/response headers
 - <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

Cross-Origin Resource Sharing (CORS)



CORS error

- The browser is not capable of rendering the data if Cross-Origin request is blocked by the server
 - i.e., when request origin does not match any allowed origins
 - **no errors if server returns: Access-Control-Allow-Origin: ***
 - then server allows all requests
- Check the reason from Developer Tools: Network traffic viewer